DevOps Portfolio Project – End-to-End Deployment

Andres Cepeda | DevOps Candidate

Project Overview

This project demonstrates a complete end-to-end DevOps workflow, starting from infrastructure provisioning to production deployment with CI/CD automation. The objective was to build, deploy, troubleshoot, containerize, and optimize a web application using industry-standard tools.

Final Production Architecture

Git → GitHub → Netlify → Internet (Global CDN + HTTPS)

Phase 1 – Infrastructure Setup (AWS EC2)

Provisioned an Amazon Linux EC2 instance and configured remote access via SSH.

Command: ssh -i key.pem ec2-user@IP

Phase 2 – Web Server Configuration (NGINX)

Installed and configured NGINX as a reverse proxy to handle incoming traffic.

Key commands:

sudo yum update -y

sudo yum install nginx -y

sudo systemctl start nginx

sudo systemctl enable nginx

Phase 3 – Application Layer (Flask)

Developed a Flask-based web application with HTML templates and static assets.

Test command: python3 app.py

Phase 4 – Application Server (Gunicorn)

Deployed Gunicorn as a WSGI server to run the Flask application in a production-like environment.

pip3 install gunicorn

gunicorn --bind 127.0.0.1:5000 app:app

Phase 5 – Troubleshooting & Debugging

Diagnosed and resolved real-world issues such as 502 errors, incorrect working directories, and port conflicts.

Example fix: sudo pkill gunicorn

Phase 6 – Service Management (systemd)

Configured Gunicorn to run as a background service, ensuring high availability and automatic restart.

sudo systemctl daemon-reload

sudo systemctl start webcv

sudo systemctl enable webcv

Phase 7 – Containerization (Docker)

Containerized the application using Docker and orchestrated services using Docker Compose.

docker-compose up -d --build

Phase 8 – Version Control (Git & GitHub)

Initialized a Git repository and pushed code to GitHub for version control and collaboration.

git init → git add . → git commit → git push

Phase 9 – CI/CD Deployment (Netlify)

Configured automatic deployments triggered by GitHub pushes using Netlify.

Resolved deployment issues by restructuring project:

mv templates/index.html .

Phase 10 – Domain & DNS Configuration

Purchased a custom domain and configured DNS records in Namecheap to point to Netlify infrastructure.

A Record: @ → 75.2.60.5

A Record: @ → 99.83.190.102

CNAME: www → netlify.app

Phase 11 – Security (HTTPS)

Enabled HTTPS using Let's Encrypt via Netlify, ensuring secure communication over TLS.

Technical Skills Demonstrated

Linux Administration, AWS EC2, NGINX, Gunicorn, Flask, Docker, Git, GitHub, CI/CD Pipelines, DNS Configuration, Domain Management, HTTPS (SSL/TLS).

Interview Summary

Designed and deployed a full-stack web application using AWS and Docker, later migrating to a cloud-native CI/CD pipeline using GitHub and Netlify. Configured custom domain routing,

DNS records, and automatic HTTPS provisioning for a production-ready deployment.

Deployment Workflow

git add . → git commit -m 'update' → git push

Automatic deployment triggered via Netlify CI/CD.